Things get trickier if you're making a lot of requests to an API using JavaScript, and you're posting JavaScript Object Notation (JSON) objects rather than form data. In these cases, you won't be able to send the verification token as part of a form (because you're sending JSON), so you'll need to add it as a header in the request instead. Microsoft's documentation "Prevent Cross-Site Request Forgery (XSRF/ CSRF) attacks in ASP.NET Core" contains an example of adding the header in JavaScript and validating it in your application. See http://mng.bz/XNNa.

TIP If you're not using cookie authentication and instead have a single-page application (SPA) that sends authentication tokens in a header, the good news is that you don't have to worry about CSRF at all! Malicious sites can send only cookies, not headers, to your API, so they can't make authenticated requests.

Generating unique tokens with the data protection APIs

The antiforgery tokens used to prevent CSRF attacks rely on the ability of the framework to use strong symmetric encryption to encrypt and decrypt data. Encryption algorithms typically rely on one or more keys, which are used to initialize the encryption and to make the process reproducible. If you have the key, you can encrypt and decrypt data; without it, the data is secure.

In ASP.NET Core, encryption is handled by the data protection APIs. They're used to create the antiforgery tokens, encrypt authentication cookies, and generate secure tokens in general. Crucially, they also control the management of the key files that are used for encryption. A *key file* is a small XML file that contains the random key value used for encryption in ASP.NET Core apps. It's critical that it's stored securely. If an attacker got hold of it, they could impersonate any user of your app and generally do bad things!

The data protection system stores the keys in a safe location, depending on how and where you host your app: